

TECHNICAL REQUIREMENTS DOCUMENT

Emergency Response Voice Agent

Technical architecture, implementation requirements, APIs, data model, agent runtime, security, testing, and deployment specification

Version	1.0
Status	Engineering Baseline
Target	AssemblyAI Voice Agent Hackathon
Primary Client	Web / Mobile-responsive Citizen App
Operations Client	Mission Control Center
Voice Platform	AssemblyAI

Engineering principle: The AI must never fabricate emergency actions. All consequential operations must pass through explicit backend tools, validation, authorization, and auditable state transitions.

1. Technical Overview

The Emergency Response Voice Agent is a realtime conversational system with two primary clients: a citizen-facing voice interface and an operator-facing Mission Control Center. The system converts natural speech into structured incident data, dynamically gathers missing information, applies controlled triage logic, and exposes safe operational actions through explicit tools.

1.1 High-Level Architecture

Citizen / Operator Voice → AssemblyAI realtime voice layer → Agent Orchestrator → Safety + State + Tool layer → Incident Service → PostgreSQL / Realtime Events → Citizen UI + Mission Control Center.

1.2 Component Responsibilities

Component	Responsibilities
Citizen Web App	Voice reporting, status, location, incident details, guidance, report history.
Mission Control UI	Map, incident queue, detail view, responder status, transcript, operator actions, voice commands.
Voice Gateway	Audio session lifecycle, AssemblyAI connection, turn/interrupt events, transcript events.
Agent Orchestrator	Conversation state, intent, missing information, tool selection, response generation.
Safety Engine	Policy checks, risk constraints, confirmation rules, unsafe-response blocking.
Incident Service	Create/read/update incidents, state machine, duplicate checks, audit events.
Location Service	GPS capture, reverse geocoding, location confidence and normalization.
Notification Service	Operator alerts and simulated escalation notifications.
RAG Service	Retrieval of approved safety procedures and site-specific documentation.
Database	Persistent incidents, conversations, events, users, responders and locations.

2. System Architecture

The system should be event-driven around an Incident domain object. Voice interactions and operator actions update the same canonical incident state, preventing the citizen UI and Mission Control Center from developing separate versions of the truth.

2.1 Logical Flow

Citizen: microphone → AssemblyAI → transcript → agent → structured facts → safety validation → tool call → incident DB → realtime event → UI update → voice response.

Operator: microphone/text → AssemblyAI or command parser → operator agent → authorization → tool call → incident DB → audit event → realtime dashboard update.

2.2 Realtime Event Bus

Use WebSockets for the hackathon MVP. Events should be emitted for incident.created, incident.updated, incident.status_changed, incident.severity_changed, responder.assigned, transcript.appended, notification.created, and system.health_changed.

3. Recommended Technology Stack

Category	Technology	Notes
Frontend	Next.js + React + TypeScript	Responsive citizen UI and Mission Control Center.
Styling	Tailwind CSS	Fast implementation of consistent light/dark command-center components.

Category	Recommended Technology	Notes
Backend	Python FastAPI	Strong fit for agent orchestration, validation, APIs and async services.
Realtime	WebSockets	Low-latency incident and transcript updates.
Voice	AssemblyAI	Realtime speech recognition and voice-agent capabilities.
LLM	Tool-calling LLM	Reasoning, extraction, question selection and command interpretation.
Database	PostgreSQL	Canonical operational data store.
Vector Search	pgvector	Simple RAG implementation without another database.
Maps	Mapbox / Google Maps equivalent	Location visualization and geocoding.
Auth	JWT / managed auth	Operator authentication and role-based access.
Deployment	Vercel + Render/Railway/AWS equivalent	Separate frontend and API deployment acceptable for MVP.

4. Voice System Requirements

The voice system is the primary technical differentiator and must feel conversational rather than request/response based.

4.1 Required Voice Behaviors

- Stream microphone audio rather than waiting for a complete recording.
- Display live connection and listening states.
- Detect user turns and agent turns.
- Allow interruption while the agent is speaking.
- Stop or supersede an active agent response when the user interrupts.
- Maintain conversation context across the session.
- Keep a text transcript synchronized with the audio conversation.
- Gracefully recover from temporary voice/network failures.
- Provide a text-input fallback when microphone access fails.

4.2 Voice Session State

IDLE → CONNECTING → LISTENING → THINKING → SPEAKING → INTERRUPTED → LISTENING → ENDED

5. Agent Runtime

The agent should be implemented as a controlled orchestrator rather than a free-form autonomous model. The model decides what information is missing and which tool is appropriate; deterministic backend services validate and execute actions.

5.1 Agent State

conversation_id, incident_id, incident_type, severity, confidence, location, people_affected, injuries, hazards, known_facts, missing_critical_facts, last_user_intent, session_status.

5.2 Agent Decision Loop

1. Receive transcript event → 2. Update conversation state → 3. Extract facts → 4. Classify intent/type → 5. Determine severity → 6. Identify critical missing information → 7. Choose next question or tool → 8. Validate action → 9. Execute tool → 10. Generate concise spoken response → 11. Persist event.

5.3 Agent Tools

```
get_current_location(), create_incident(), update_incident(), get_incident(), search_nearby_incidents(), find_duplicate_incidents(),
get_emergency_protocol(), notify_operator(), assign_responder(), update_incident_status(), get_incident_statistics(),
generate_incident_report()
```

6. Triage and Classification

Classification should combine model extraction with deterministic validation. The model may propose a classification, but the backend stores normalized enum values.

6.1 Incident Types

MEDICAL, FIRE, ROAD_ACCIDENT, NATURAL_DISASTER, INDUSTRIAL, GAS_LEAK, ELECTRICAL, PUBLIC_SAFETY, OTHER

6.2 Severity

LOW, MEDIUM, HIGH, CRITICAL

Severity must be explainable internally through extracted evidence. Example: an unresponsive person + trapped person should be eligible for a critical classification, subject to the configured safety policy.

7. Incident State Machine

REPORTED → VERIFYING → TRIAGED → DISPATCH_PENDING → RESPONDER_ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED

Terminal/alternative state: REPORTED → FALSE_OR_DUPLICATE.

8. Backend API Requirements

Method	Endpoint	Description
POST	/api/v1/incidents	Create incident
GET	/api/v1/incidents	List/filter incidents
GET	/api/v1/incidents/{id}	Get incident details
PATCH	/api/v1/incidents/{id}	Update allowed incident fields
POST	/api/v1/incidents/{id}/status	Change incident status
POST	/api/v1/incidents/{id}/assign	Assign responder
POST	/api/v1/incidents/{id}/merge	Merge duplicate incident
GET	/api/v1/incidents/{id}/conversation	Get transcript
POST	/api/v1/voice/session	Create voice session
POST	/api/v1/location/resolve	Resolve user-described location
GET	/api/v1/responders	List responders
POST	/api/v1/notifications	Create notification
GET	/api/v1/metrics/overview	Mission-control KPI summary
GET	/api/v1/health	Service health check

9. Database Schema

Table	Columns
incidents	id, type, severity, confidence, status, description, location_id, people_affected, created_at, updated_at
incident_facts	id, incident_id, fact_type, value, confidence, source, created_at
conversations	id, incident_id, channel, started_at, ended_at
transcript_messages	id, conversation_id, speaker, text, timestamp, confidence

Entity	Attributes
incident_events	id, incident_id, event_type, actor_id, payload, created_at
locations	id, latitude, longitude, accuracy, address, source, confidence
responders	id, name, team, status, current_location, last_seen_at
assignments	id, incident_id, responder_id, assigned_at, status
notifications	id, incident_id, recipient, channel, status, created_at
users	id, role, display_name, auth_provider, created_at

9.1 Example Incident JSON

```
{ "id": "INC-1042", "type": "ROAD_ACCIDENT", "severity": "CRITICAL", "confidence": 0.92, "status": "RESPONDER_ASSIGNED",
  "people_affected": 3, "injured": 2, "unresponsive": 1, "trapped": 1, "hazards": ["SMOKE_REPORTED"], "location": { "lat": 28.67, "lng":
  77.43, "accuracy_m": 18 } }
```

10. Mission Control Center Requirements

The desktop-first operations dashboard should prioritize rapid situational awareness. The light-theme design should use strong severity hierarchy without making the interface visually aggressive.

- Top KPI cards: Critical, High, Medium, Low, Total Incidents, Active Responders.
- Live incident map with incident, responder, hospital, traffic, weather and risk-zone layers where data exists.
- Incident filters by severity, type, status, location and time.
- Live incident feed with compact operational rows.
- Active incident panel with actions and structured facts.
- Conversation transcript with live state.
- Responder assignment and status.
- Quick actions such as search nearby, show critical incidents, responders, and generate report.
- Voice command panel for supported operator commands.
- System health panel for voice, location, database, notifications and AI agent.

11. Citizen Application Requirements

The mobile-first citizen interface should make the emergency action immediately discoverable and reduce unnecessary navigation.

- Large primary 'Report an Emergency' voice control.
- Emergency category shortcuts as an optional alternative to free-form voice.
- Visible active conversation state.
- Live transcript during an emergency session.
- Automatic location capture with explicit permission.
- Submitted incident status and incident ID.
- Recent reports and report details.
- Safety resources and general guidance.
- Prominent emergency-service fallback such as the local emergency number; never imply that the app itself contacted emergency services unless it actually did.
- Text-input fallback.

12. Multi-Witness and Duplicate Detection

A candidate duplicate is generated when reports are close in time and space and have semantically similar incident descriptions. The system should not automatically merge high-severity incidents without operator review.

12.1 Suggested Scoring

```
duplicate_score = weighted(time_proximity, geo_proximity, incident_type_match, semantic_similarity).
```

Recommended initial weights: geography 30%, time 20%, type 20%, semantic similarity 30%. These are implementation defaults and should be tuned using synthetic test scenarios.

13. RAG / Safety Knowledge

RAG is optional for MVP but recommended for the polished version. Only approved documents should be indexed. Retrieval results should include source metadata and document version.

- Chunk approved procedures by operational step.
- Attach document name, version, jurisdiction/site and effective date.
- Retrieve only when the incident type warrants procedural guidance.
- Never allow retrieved text to override hard safety policies.
- Keep guidance concise for voice output.

14. Security, Privacy and Safety

Because emergency data can be highly sensitive, security must be designed into the system even for the hackathon prototype.

- Use TLS for all network traffic.
- Protect API keys and secrets with environment/secret management; never ship them to the browser.
- Authenticate Mission Control users.
- Apply role-based authorization to operator/responder actions.
- Validate every tool call server-side.
- Log consequential actions in an immutable-style audit trail.
- Minimize personally identifiable information.
- Do not retain raw audio longer than required for the product workflow.
- Prefer synthetic/demo identities and incidents during judging.
- Never fabricate dispatch, notification, responder assignment, or emergency-service contact.

15. Failure Handling

Failure	Recommended Response
Microphone denied	Show text fallback and emergency-number option.
AssemblyAI unavailable	Display service degraded state; allow text incident creation.
LLM timeout	Retry once, then fall back to deterministic incident form.
Location unavailable	Ask user to describe location; mark location confidence accordingly.
Database failure	Do not claim incident was saved; show retry state.
Notification failure	Record failure and surface it to operator.
Tool failure	Return explicit failure to agent; agent must not claim success.
Duplicate ambiguity	Flag for operator review rather than auto-merging.

16. Testing Strategy

- Unit tests: classification normalization, severity rules, state transitions, authorization, location normalization.
- Integration tests: AssemblyAI session, LLM tool calling, database persistence, WebSocket events, notification flow.
- Conversation tests: scripted emergency scenarios with expected extracted facts and required follow-up questions.
- Interruption tests: user interrupts during agent speech; verify the previous response is cancelled/superseded.
- Failure tests: network loss, voice service failure, invalid tool response, database timeout.
- Security tests: unauthorized operator actions, malformed IDs, injection attempts, secret exposure.

- Demo acceptance tests: complete road-accident scenario from citizen voice report to Mission Control status update.

16.1 Minimum Demo Scenarios

Scenario	Expected Outcome
Road accident, critical	Incident created as critical; location and injuries extracted; command center updates.
Fire report	Fire classification and hazard questions triggered.
Medical collapse	Critical missing information questions triggered; no diagnosis claims.
Two witnesses, same crash	Potential duplicate surfaced to operator.
Operator voice query	Supported incident query returns correct data.
Operator status update	Authorized tool updates incident and audit log.

17. Observability and Performance

- Measure end-to-end voice latency and each major stage: STT, agent reasoning, tool execution, TTS.
- Log correlation ID across voice session, agent turn, tool call and incident event.
- Track tool failures, transcript errors, dropped sessions and WebSocket disconnects.
- Mission Control should update incident state in near real time.
- Keep structured logs free of unnecessary sensitive content.

18. Deployment Architecture

```

Browser / Mobile Web
↓ HTTPS / WebSocket
Frontend (Next.js)
↓ HTTPS
API + Agent Service (FastAPI)
├─ AssemblyAI
├─ LLM provider
├─ PostgreSQL / pgvector
├─ Maps / Geocoding
└─ Notification provider / mock adapter

```

18.1 Environment Separation

- development: local synthetic data and test credentials
- staging/demo: isolated database, synthetic emergency incidents, production-like logging
- production future: authorized emergency integrations, hardened security, retention policies, monitoring and formal safety review

19. Implementation Plan

Sprint	Tasks
Sprint 1	Repository setup, database schema, authentication, base citizen UI, base Mission Control UI.
Sprint 2	AssemblyAI realtime voice session, transcript UI, agent runtime, basic tool calling.
Sprint 3	Incident extraction, classification, severity, dynamic questions, incident state machine.
Sprint 4	Mission Control map/feed/detail, WebSocket updates, responder assignment, audit events.
Sprint 5	Multi-witness duplicate detection, voice operator commands, safety validation, failure handling.

Iteration	Deliverables
Sprint 6	Testing, latency tuning, UI polish, demo scenario, deployment and documentation.

20. Technical Acceptance Criteria

- A citizen can start a voice session from the mobile UI and speak naturally.
- AssemblyAI transcript events appear in the interface in realtime.
- The agent can classify at least road accident, fire, medical, and other scenarios.
- The agent asks context-dependent follow-up questions.
- The system creates a normalized incident with severity, location and extracted facts.
- Mission Control receives the incident without requiring a manual refresh.
- An operator can inspect the transcript and structured incident details.
- At least two operator voice commands execute through validated tools.
- Every consequential action creates an audit event.
- The system never claims a failed or simulated action succeeded.
- The full road-accident demo can be completed reliably using synthetic data.

21. Reference Agent Workflow

LISTEN → TRANSCRIBE → UNDERSTAND → EXTRACT → CLASSIFY → IDENTIFY MISSING FACTS → ASK / ACT → VALIDATE → PERSIST → UPDATE COMMAND CENTER → RESPOND

The technical design intentionally separates probabilistic AI reasoning from deterministic operational execution. This is the key engineering constraint for making the project credible, testable, and safe.

Appendix A. Example Operator Commands

“Show me all critical incidents.”
 “What’s happening at incident 1042?”
 “How many incidents are waiting for responders?”
 “Assign the nearest available responder to incident 1042.”
 “Mark incident 1042 as responder assigned.”
 “Generate the incident report for 1042.”

Appendix B. MVP vs Future Scope

MVP	Future
Realtime voice reporting	Telephony integration
Mock escalation	Authorized emergency dispatch integrations
Single orchestrating agent	Multi-agent orchestration
Basic duplicate detection	Large-scale incident correlation
Web geolocation	Wearables / IoT / CCTV
Synthetic safety knowledge	Jurisdiction-specific production knowledge base

End of Technical Requirements Document